Archivos, formatos y rutas

Introducción a la Ciencia de Datos — Universidad de Montevideo

Lo que vimos hasta ahora en el curso

	Tema	Qué hicimos
	Medición y encuestas	operacionalización, niveles de medición, diseño de un cuestionario
	Carga de datos	leer archivos en distintos formatos; rutas y directorio de trabajo
	Perfilado y fechas	filas, columnas, faltantes, fechas y agregación con los datos abiertos de aire y tránsito
	Gráficos	las partes de un gráfico, más de una serie, colores por grupo
	Unión de tablas	merge de ozono con PM2.5 por la hora
	Series de tiempo	grilla regular, rezago y autocorrelación
	IA y OCR	leer tickets: la imagen se manda como bytes a la API de Gemini
	Variables y modelo	variables nuevas, correlación y la recta, con la tabla de tránsito y aire

Hoy volvemos a la base: **qué hay dentro de un archivo, y cómo lo encuentra el programa.**

Qué es un archivo

El bit y el byte 💡

Bit (*binary digit*). La unidad mínima de información: vale 0 o 1. En el hardware es algo que está en uno de dos estados, como un interruptor apagado o encendido.

Byte. Un grupo de 8 bits. Con 8 lugares que valen 0 o 1 hay 2^8 = 256 combinaciones distintas: de `00000000` a `11111111`, es decir, de 0 a 255.

Cada lugar vale el doble que el de su derecha, como en el sistema decimal cada lugar vale diez veces más:

Valor del lugar	128	64	32	16	8	4	2	1
Bits	0	**1**	0	0	0	0	0	**1**

`01000001` = 64 + 1 = **65**. En la tabla ASCII, 65 es la letra **A**. A

El hexadecimal: un byte en dos cifras

Escribir `01000001` es largo. El **sistema hexadecimal** usa 16 cifras: `0 1 2 3 4 5 6 7 8 9 A B C D E F`, donde A vale 10 y F vale 15.

Cada cifra hexadecimal reemplaza exactamente 4 bits, así que un byte siempre se escribe con dos cifras:

Bits	Se parte en	Hexadecimal	Decimal
`01000001`	`0100` · `0001`	`41`	65
`11000011`	`1100` · `0011`	`C3`	195
`11111111`	`1111` · `1111`	`FF`	255

Por eso en esta clase los bytes aparecen como `41`, `C3` o `FF`. En Python, un byte que no es un carácter se muestra como `\xc3`: la `\x` avisa que lo que sigue es hexadecimal.

Cuánto ocupa un archivo 

Unidad	Bytes	Un ejemplo del curso
byte	1	una letra sin tilde en un CSV
kilobyte (KB)	1000	`ventas.csv`: 70 bytes; `transito_aire_junio2024.csv`: 131 KB
megabyte (MB)	1 000 000	una foto de 1000 × 1000 píxeles sin comprimir: 3 MB
gigabyte (GB)	1 000 000 000	una película, o un año de datos minutales de varias estaciones

Windows cuenta de a 1024 en lugar de 1000 (1 KB = 1024 bytes), así que los tamaños que muestra pueden diferir un poco de estos.

Un archivo es una secuencia de bytes con un nombre

Archivo. Secuencia finita de bytes guardada en un disco, con un nombre y una ubicación (la carpeta donde está). El archivo no sabe qué significan sus bytes: eso lo decide el programa que lo lee.

Los primeros 12 bytes de `ventas.csv`:

66
65
63
68
61
2C
6C
6F
63
61
6C
2C

Así lo ve la computadora: solo números. 🤖 Para saber si son letras, píxeles o celdas de una planilla, hace falta una regla que diga cómo leerlos.

Todo archivo es una fila de bytes: cambia la regla para leerlos

	Archivo	Primeros bytes	Qué son
	`ventas.csv`	`66 65 63 68 61 2C`	las letras «fecha,»
	`saludo.py`	`70 72 69 6E 74 28`	las letras «print(»
	`ventas.xlsx`	`50 4B 03 04`	«PK»: la firma de un archivo ZIP
	`ejercicios.pdf`	`25 50 44 46`	«%PDF»: la firma de un PDF
	`foto.jpg`	`FF D8 FF E0`	la firma de una imagen JPG
	`tono.wav`	`52 49 46 46`	«RIFF»: la firma de un audio WAV

Los primeros bytes de muchos formatos son una **firma**: le dicen al programa qué tipo de archivo es. Después vienen letras, píxeles, celdas o sonido, según el formato.

Texto plano: cada byte es un carácter

Archivo de texto plano. Archivo cuyos bytes representan solo caracteres, según una codificación (una tabla que asigna un número a cada carácter). No guarda formato: ni negrita, ni colores, ni imágenes.



Son de texto plano: `.txt` `.csv` `.py` `.R` `.qmd` `.md` `.json` `.html`. Se pueden abrir y leer con cualquier editor de texto.

Archivo binario: los bytes no son caracteres

Archivo binario. Archivo cuyos bytes siguen una estructura propia del formato: números, imágenes comprimidas, partes de otros archivos. Solo lo lee bien el programa que conoce esa estructura.

`ventas.xlsx` abierto con el Bloc de notas: 🗑️

```
PK   ? a]I:O  [Content_Types].xmlËnÂ0
E÷ý
ÈÛ*1tQUEË©ô\{B,ÛòþŸðP[Q M¬dîÜsÇ<--
```

Un `.xlsx` es una carpeta de archivos comprimida (por eso empieza con PK, como un ZIP). El resto no son caracteres, y el editor los muestra como basura.

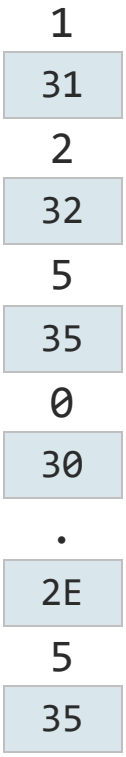
Los mismos datos, 3 columnas y 2 filas:

70 bytes en `ventas.csv`

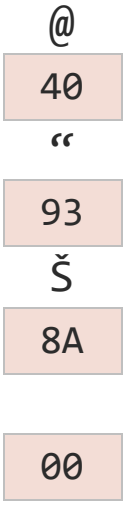
5360 bytes
en `ventas.xlsx`

Texto o binario: cómo se guarda el número 1250,5

En un archivo de texto se guardan los caracteres que se ven: «1», «2», «5», «0», «.», «5». Son 6 bytes, uno por carácter:



En un archivo binario se guarda el número mismo, con la regla que usa la computadora para hacer cuentas: un número decimal ocupa siempre 8 bytes.



00

00

00

00

Arriba de cada byte: lo que muestra el Bloc de notas si lo lee como texto. 🗑️

- En texto, al leer, el programa tiene que **traducir caracteres a números**. Ahí aparecen los problemas de separador, decimal y codificación.
- En binario el número ya está listo, pero hay que **saber el formato**: dónde empieza cada dato y cuántos bytes ocupa.

Un ejemplo de R: el formato .rds 📦

```
ventas <- read.csv("ventas.
saveRDS(ventas, "ventas.rds
ventas2 <- readRDS("ventas.
identical(ventas, ventas2)
```

`ventas.rds` empieza con `1F 8B`: la firma de un archivo comprimido con gzip. 🗜️

Sin comprimir (`compress = FALSE`), adentro está la columna `monto`:

vector de números decimales (tipo 14)

```
00
00
00
0E
```

con 2 valores

```
00
00
00
02
```

1250,5

```
40
93
8A
00
00
00
```

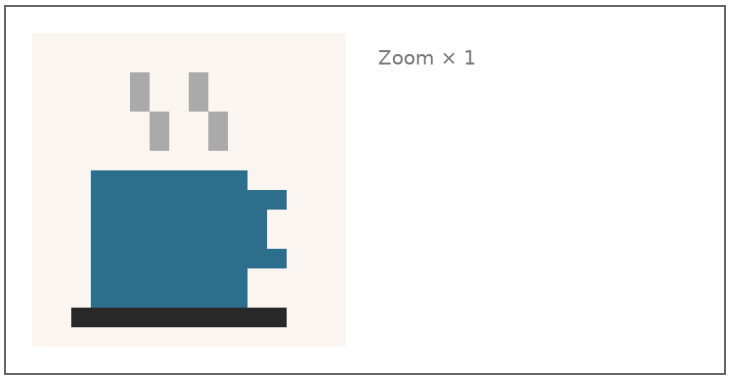
00
00

980

40
8E
A0
00
00
00
00
00

Gana: se lee de nuevo idéntica, con los tipos de cada columna, sin separador ni decimal. **Pierde:** solo la lee R (Python necesita un paquete como `pyreadr`), no se puede abrir con el Bloc de notas y Git no muestra qué cambió.

Una imagen también es una secuencia de bytes



Píxel. Cada punto de una imagen. Su color se guarda con tres bytes: cuánto rojo, cuánto verde y cuánto azul tiene, de `00` a `FF`.

Un JPG comprime los píxeles: abierto con el Bloc de notas, es basura.

En la clase de OCR, la foto del ticket se abrió así: 📜

```
archivo = open(RUTA + FOTO, "rb"
```

`"rb"`: *read binary*, leer los bytes sin tratarlos como texto.

Texto plano y binario, lado a lado

	Texto plano	Binario
Qué guardan los bytes	Caracteres, según una codificación	Una estructura propia del formato
Para leerlo hace falta	Saber la codificación	Un programa que conozca el formato
Con el Bloc de notas	Se lee y se puede editar	Se ve basura; si se guarda, se rompe
En Git (GitHub Desktop)	Muestra qué líneas cambiaron	Solo dice que el archivo cambió
En Python	`open(ruta, "r", encoding="utf-8")`	`open(ruta, "rb")`
Ejemplos	`.txt` `.csv` `.py` `.R` `.qmd` `.json`	`.xlsx` `.docx` `.pdf` `.jpg` `.zip`

Criterio: un archivo es de texto plano si, leídos sus bytes con una codificación, todo lo que aparece son caracteres y el texto tiene sentido.

Actividad: seis archivos con el Bloc de notas



Abrir cada archivo con el Bloc de notas: clic derecho → *Abrir con* → *Bloc de notas*. En Mac, con TextEdit.

Un script de Python es un archivo de texto: se puede escribir y corregir con cualquier editor, y Python lo lee y lo ejecuta.

La extensión

La extensión es parte del nombre, no del contenido

Extensión. Las letras que siguen al último punto del nombre: en `ventas.csv`, la extensión es `csv`. El sistema operativo la usa para elegir con qué programa abrir el archivo. Cambiarla no cambia ningún byte.

Antes		Después	Resultado
`ventas.csv` (70 bytes de texto)		`ventas.txt` (los mismos 70 bytes)	**Sigue siendo el mismo CSV** ✅
`ventas.xlsx` (5360 bytes binarios)		`falso.csv` (los mismos 5360 bytes)	**No es un CSV:** `UnicodeDecodeError` ❌

Windows oculta las extensiones 🙈. Para verlas: Explorador de archivos → *Vista* → *Mostrar* → *Extensiones de nombre de archivo*. En Mac: Finder → *Ajustes* → *Avanzado* → *Mostrar todas las extensiones*.

El CSV

Un CSV es texto plano con tres convenciones

CSV (*comma-separated values*). Archivo de texto plano que representa una tabla: una fila por línea, la primera con los nombres de las columnas. Para leerlo hay que saber tres cosas:

- **Separador:** el carácter que separa las columnas (coma o punto y coma).
- **Decimal:** el carácter que marca los decimales (punto o coma).
- **Codificación:** la tabla que pasa de caracteres a bytes.

```
fecha,local,monto
03/01/2024,Pocitos,1250.50
04/01/2024,Unión,980.00
```

Coma y punto: la convención de la norma

```
fecha;local;monto
03/01/2024;Pocitos;1250,50
04/01/2024;Unión;980,00
```

Punto y coma y coma decimal: Excel en Uruguay

UY

La norma (RFC 4180) fija la coma como separador. Donde la coma es el signo decimal, Excel guarda con punto y coma. El archivo no dice cuál usa: hay que abrirlo y mirar. 👀

La codificación

ASCII: la primera tabla común, sin tildes

Codificación. Regla que asigna a cada carácter un número, y dice cómo se escribe ese número en bytes. Sin una regla compartida, dos computadoras no pueden intercambiar texto.

ASCII (*American Standard Code for Information Interchange*, 1963) usa 7 bits: 128 números, del 0 al 127. Alcanza para el inglés: letras sin tilde, cifras, signos de puntuación y caracteres de control, como el salto de línea. us

Carácter	Número	Byte
A	65	41
a	97	61
0	48	30
,	44	2C
salto de línea	10	0A
ó, ñ, á	**no está**	—

El número de cada carácter se escribe en un byte. Como un byte tiene 8 bits, quedan libres los números del 128 al 255.

El mismo byte, tres letras distintas

Cada región usó los números del 128 al 255 para sus propias letras. Así nacieron las **páginas de códigos**: una tabla distinta por idioma.

Byte		Letra	Página de códigos
F3		ó	**Latin-1** (ISO 8859-1, 1987): Europa occidental. Windows usa una variante, Windows-1252, que el Bloc de notas llama «ANSI».
F3		у	**Windows-1251**: cirílico (ruso, ucraniano, búlgaro).
F3		σ	**ISO 8859-7**: griego.

Byte	Letra	Página de códigos

Un archivo no dice con qué página de códigos se escribió. Abierto con otra, muestra otras letras: es el origen de los textos con caracteres raros. 😯

Unicode: un número para cada carácter de cada idioma

Unicode es una sola lista de caracteres para todos los sistemas de escritura. La publica el Consorcio Unicode desde 1991 (versión 1.0) y hoy tiene más de 150 000 caracteres: alfabetos, ideogramas, símbolos matemáticos y emojis. 🌍

Punto de código. El número que Unicode le asigna a un carácter. Se escribe U+ seguido del número en hexadecimal. Los primeros 128 coinciden con ASCII.

Carácter	Punto de código
A	U+0041
ó	U+00F3
ñ	U+00F1
€	U+20AC
中	U+4E2D
😀	U+1F600

Unicode dice qué número tiene cada carácter, pero no cómo se guarda ese número en bytes. Eso lo resuelve una codificación de

Unicode, y la más usada es UTF-8.

UTF-8: entre uno y cuatro bytes por carácter

UTF-8 (*Unicode Transformation Format, 8 bits*) escribe cada punto de código con 1, 2, 3 o 4 bytes. La diseñaron Ken Thompson y Rob Pike en septiembre de 1992, en los laboratorios Bell, para el sistema Plan 9 🍽️. Se normalizó en 2003 (RFC 3629).

Puntos de código	Bytes	Ejemplo	Bytes del ejemplo
`U+0000` a `U+007F` (ASCII)	1	A	`41`
`U+0080` a `U+07FF`	2	ó	`C3 B3`
`U+0800` a `U+FFFF`	3	€	`E2 82 AC`
`U+10000` en adelante	4	😀	`F0 9F 98 80`

Por qué se impuso. Un texto ASCII ya es UTF-8 válido, byte por byte, así que los archivos viejos en inglés no hubo que convertirlos. Y puede escribir cualquier carácter de Unicode. Hoy lo usa el 99 % de los sitios web, y es la codificación por defecto de Python y de pandas.

Cómo la ó se convierte en C3 B3

La ó es el punto de código `U+00F3`: el número 243, que en binario es `11110011`. Para números de este tamaño, UTF-8 usa dos bytes con un molde fijo.

	Primer byte	Segundo byte
Molde	`110xxxxx`	`10xxxxxx`
243 en 11 bits	`00011`	`110011`
Resultado	`11000011` = **C3**	`10110011` = **B3**

Gris: bits fijos del molde. Azul: los bits del número.

Los bits fijos permiten reconocer los bytes: uno que empieza con `110` abre un carácter de dos bytes, y uno que empieza con `10` lo continúa. Cuando Python encuentra un byte que no encaja en el molde, como el `F3` de Latin-1, da *invalid continuation byte*.

UniÃ³n: bytes de UTF-8 leídos con Latin-1

Al revés, un archivo en Latin-1 leído como UTF-8 sí da error 💥:

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf3 in
position 59: invalid continuation byte
```

La solución es decirle a pandas la codificación:
`pd.read_csv(ruta, encoding="latin-1")`.

Regla para la codificación: primero UTF-8 🧭

Paso	Qué se hace	Qué se ve	Qué quiere de
1	`pd.read_csv(ruta)` (UTF-8, por defecto)	Las tildes y la ñ se ven bien	✅ El arch en UTF-8.
2	lo mismo que en el paso 1	`UnicodeDecodeError` 💥	El archivo en UTF-8. `encoding 1"`.
3	`pd.read_csv(ruta, encoding="latin-1")`	Aparece `Ã³`, `Ã±`, `Ã¡` 👽	El archivo UTF-8 y se Sacar el `en`

Por qué primero UTF-8. UTF-8 tiene un molde: si los bytes no encajan, da error y avisa. Latin-1 no avisa nunca, porque los 256 bytes posibles son letras válidas. Si se empieza por Latin-1, el texto puede quedar mal sin que nada lo indique.

Para ver en qué está guardado: el Bloc de notas y VS Code lo muestran abajo a la derecha (*UTF-8* o *ANSI*). **Para pasarlo a UTF-8:** Bloc de notas → *Guardar como* → Codificación *UTF-8*.

Cuatro síntomas al leer un CSV, y su causa

Síntoma	Causa	Se arregla con
🧱 La tabla queda con una sola columna	Separador	`sep=";"`
🔤 La columna monto queda como texto	Decimal	`decimal=","`
👽 Aparece UniÃ³n en lugar de Unión	Codificación	`encoding="latin-1"`
🔍 `FileNotFoundError`	Ubicación del archivo	corregir la ruta

Los tres primeros se resuelven con argumentos de `pd.read_csv`, sin modificar el archivo. El cuarto no tiene que ver con el contenido: el programa no encontró el archivo.

```
pd.read_csv("ventas_pc.csv", sep=";", decimal=",", encoding="utf-
```

Actividad: escribir un CSV a mano y romperlo 🔨

1. Abrir el Bloc de notas y escribir una tabla de cinco filas: fecha, local y monto.
2. Guardarla como `mis_ventas.csv`, en la carpeta de la clase, con codificación UTF-8.
3. Leerla con pandas y comprobar que tiene 5 filas y 3 columnas.
4. Romperla de a una cosa por vez: separador, decimal, codificación, nombre. Anotar el síntoma que aparece.
5. Arreglar cada síntoma con un argumento de `pd.read_csv`, sin volver a tocar el archivo.

   Los pasos están en `practico_archivos.py`.

Carpetas y rutas

El sistema de archivos es un árbol 🌳

```
C:/
└── Users/
    └── ana/
        └── curso/
            └── clases/
                ├── clase10/
                │   └── transito_aire_junio2024.csv
                └── clase12/
                    └── clase12_ejercicios.py
```

Carpeta (directorio). Un lugar con nombre que contiene archivos y otras carpetas. 📂

Raíz. La carpeta que contiene a todas las demás: `C:/` en Windows, `/` en Mac y Linux.

Ruta. La lista de carpetas que hay que recorrer para llegar a un archivo, separadas por `/`.

Ruta absoluta y ruta relativa

Ruta absoluta. Empieza en la raíz. Solo sirve en la computadora donde existe esa carpeta. 🏠

```
C:/Users/ana/curso/clases/clase10/transito_aire_junio2024.csv
```

Ruta relativa. Empieza en el directorio de trabajo. Desde `curso/`: 🧭

```
clases/clase10/transito_aire_junio2024.csv
```

La ruta relativa funciona en la computadora de cualquier compañero que tenga la carpeta `curso/`, esté donde esté.

El directorio de trabajo decide dónde se busca

Directorio de trabajo. La carpeta desde la que el programa resuelve las rutas relativas. En Python se ve con `Path.cwd()`; en R, con `getwd()`. Dos puntos (`..`) significan «subir una carpeta».

En VS Code, cada forma de correr usa otra carpeta

▶️ Run Python File

Corre el script en la terminal. El directorio de trabajo es la carpeta abierta en VS Code (*Archivo → Abrir carpeta*).

```
curso/
```

🧩 Por celdas `#%%` (Shift + Enter)

Corre en la ventana interactiva. El directorio de trabajo es, por defecto, la carpeta donde está el script.

```
curso/clases/clase12/
```

Regla del curso: se abre siempre la carpeta del repositorio, nunca un archivo suelto. El repositorio trae el archivo `.vscode/settings.json`, que hace que las celdas también usen la carpeta del repositorio. Así las dos formas de correr dan el mismo directorio de trabajo.

Dos líneas al comienzo de cada script ✅

```
from pathlib import Path

RUTA = "clases/clase10/transito_aire_junio2024.csv"

print(Path.cwd())            # dónde está parado el programa
print(Path(RUTA).exists())   # True si el archivo está ahí
```

- **Si dice `True` ✅**, la ruta está bien y se puede leer el archivo.
- **Si dice `False` ❌**, el problema es la ruta. Se compara lo que imprimió `Path.cwd()` con el árbol de carpetas, y se corrige la ruta o la carpeta abierta.

  Con estas dos líneas, un `FileNotFoundError` deja de ser un mensaje largo y se vuelve una pregunta con respuesta: ¿desde dónde corre el programa, y dónde está el archivo?

Fuentes

McKinney, W. (2022). *Python for Data Analysis* (3.ª ed.). O'Reilly. Cap. 6: Data Loading, Storage, and File Formats.

Wickham, H., Çetinkaya-Rundel, M. y Grolemund, G. (2023). *R for Data Science* (2.ª ed.). O'Reilly. Cap. 6: Workflow: scripts and projects.

Shafranovich, Y. (2005). *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. RFC 4180, IETF.

Yergeau, F. (2003). *UTF-8, a transformation format of ISO 10646*. RFC 3629, IETF.

The Unicode Consortium (2024). *The Unicode Standard*, versión 16.0.

Pike, R. (2003). *UTF-8 history*. Correo electrónico reproducido en doc.cat-v.org/bell_labs/utf-8_history.

W3Techs (2026). *Usage statistics of character encodings for websites*.

Python Software Foundation. *pathlib — Object-oriented filesystem paths*. Documentación de Python 3.

pandas development team. *pandas.read_csv*. Documentación de pandas.

Speaker notes